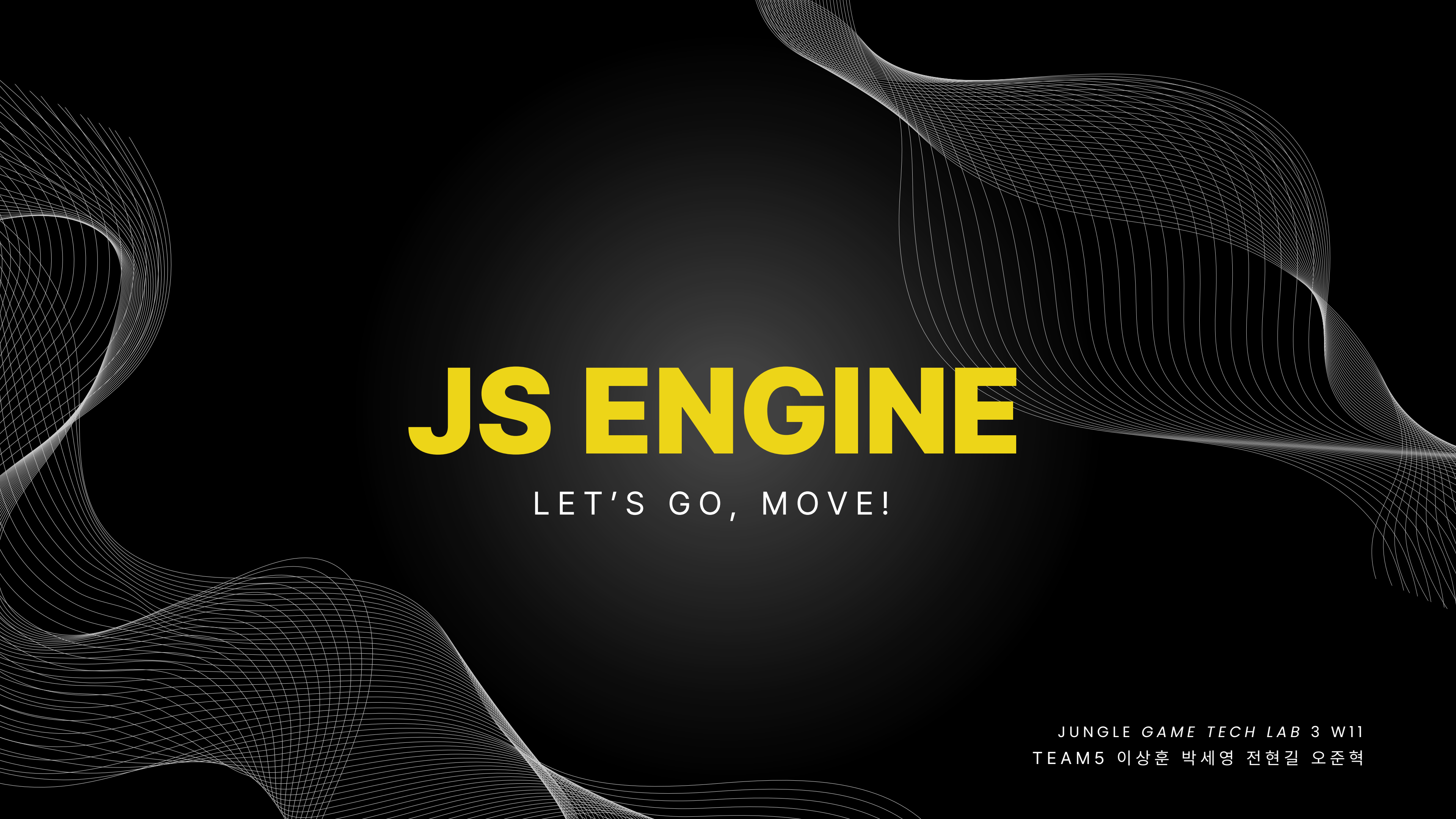




# JS ENGINE

LET'S GO, MOVE!

JUNGLE GAME TECH LAB 3 W11  
TEAM5 이상훈 박세영 전현길 오준혁



# 목차

- FBX Importer
- Animation Sequence
- Animation State Machine
- GPU Skinning
- Reflection

# 애니메이션 import, 왜 블렌더에서는 빠르고 언리얼에서는 느릴까요?

나 억울해



←  
2s

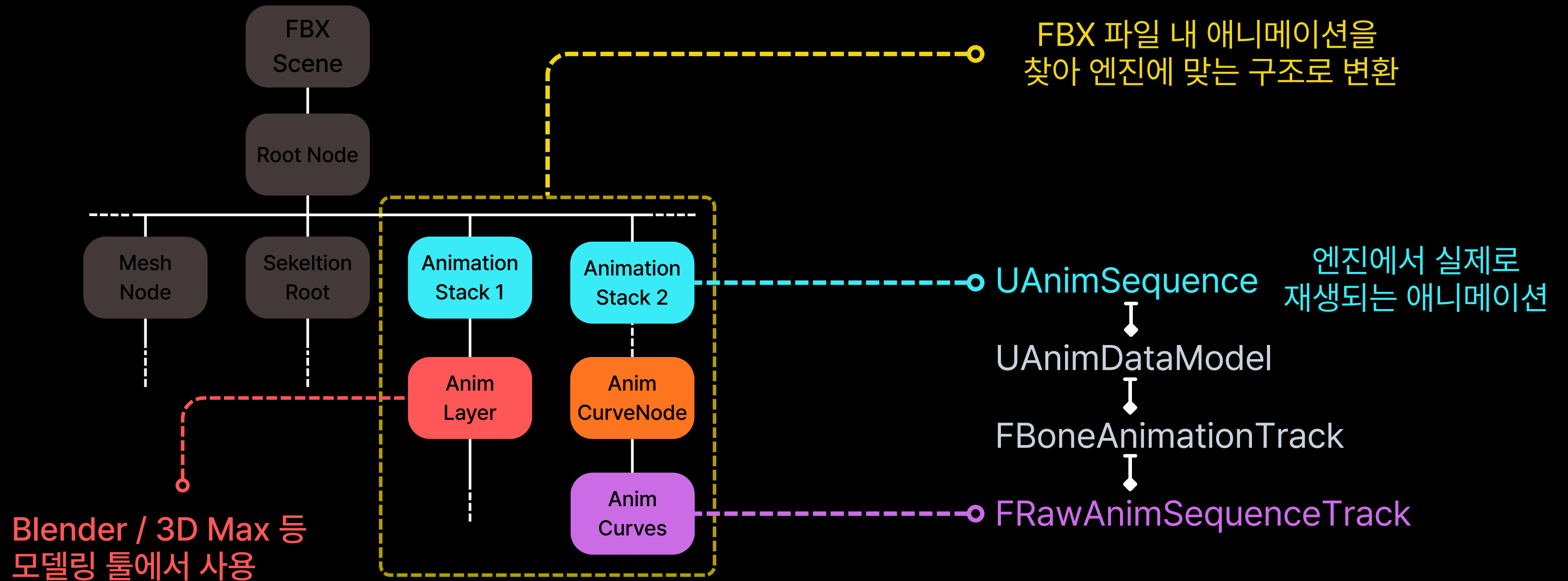


→  
2m 20s  
skeletal meshes  
animations

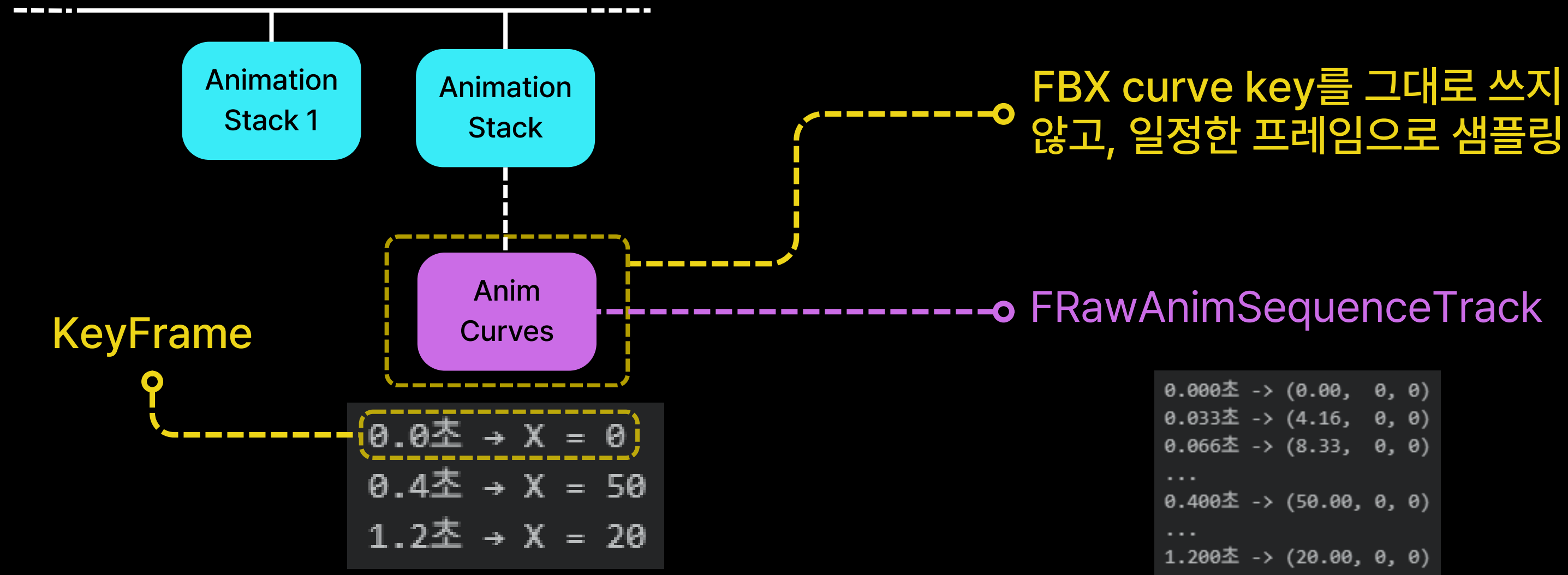


Ahri Model (375 Bones, 9,797 Vertices, 15,286 Triangles, 47 Animation Clips, 79.9MB)  
Intel(R) Core(TM) i7-14700 (20 Cores, 2.10 GHz)

# FBX Animation Importer

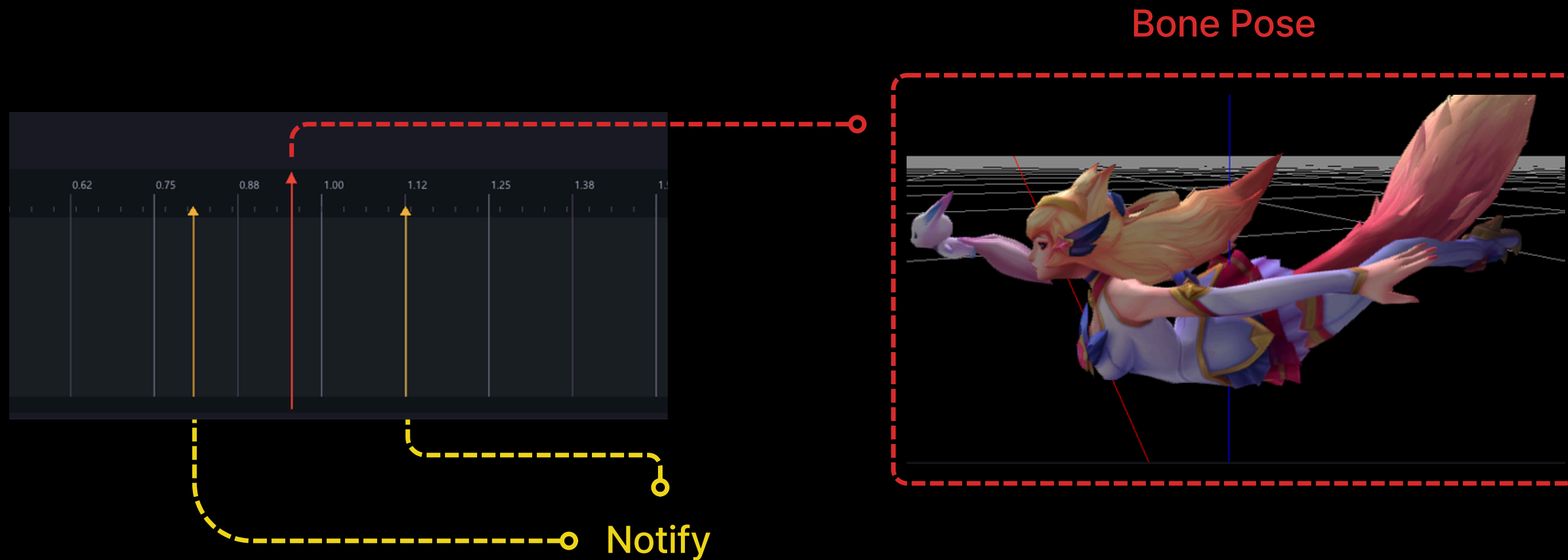


# Importer - Sampling



각 curve 마다 key 개수, key 시간, 계산 방식이 다르므로 import 시점에 일정한 간격으로 나누고 런타임 재생은 각 Track의 값 사이를 보간

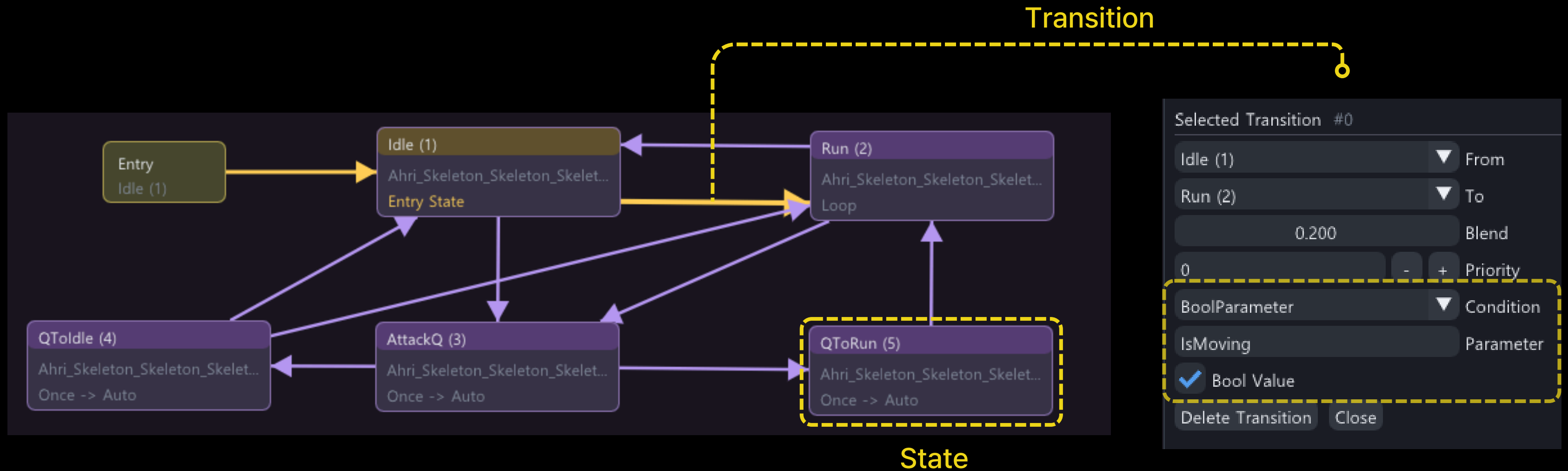
# Animation Sequence



Timeline 위에 Bone별 **키프레임**과 **Notify** 이벤트 저장  
**특정 시간의** Pose와 이벤트를 평가하는 애니메이션 데이터

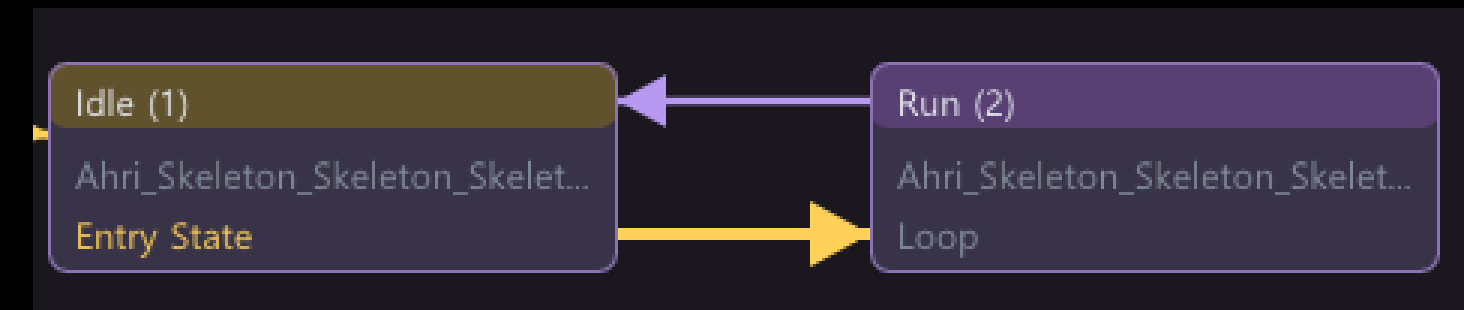


# Animation StateMachine

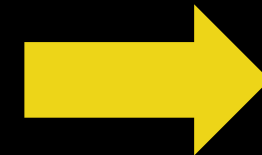


캐릭터가 현재 어떤 애니메이션인지 관리하고,  
조건에 따라 다음 상태로 전환하는 애니메이션 흐름 제어 장치

# Animation StateMachine



Blending



이전 포즈에서 다음 포즈로  
자연스럽게 넘어가게 만드는 과정

$$q(t) = (1-t)q_1 + tq_2$$

Position / Scale → Lerp

$$r = \frac{\sin\theta(1-t)}{\sin\theta}P_1 + \frac{\sin\theta t}{\sin\theta}P_2$$

Rotation → SLerp



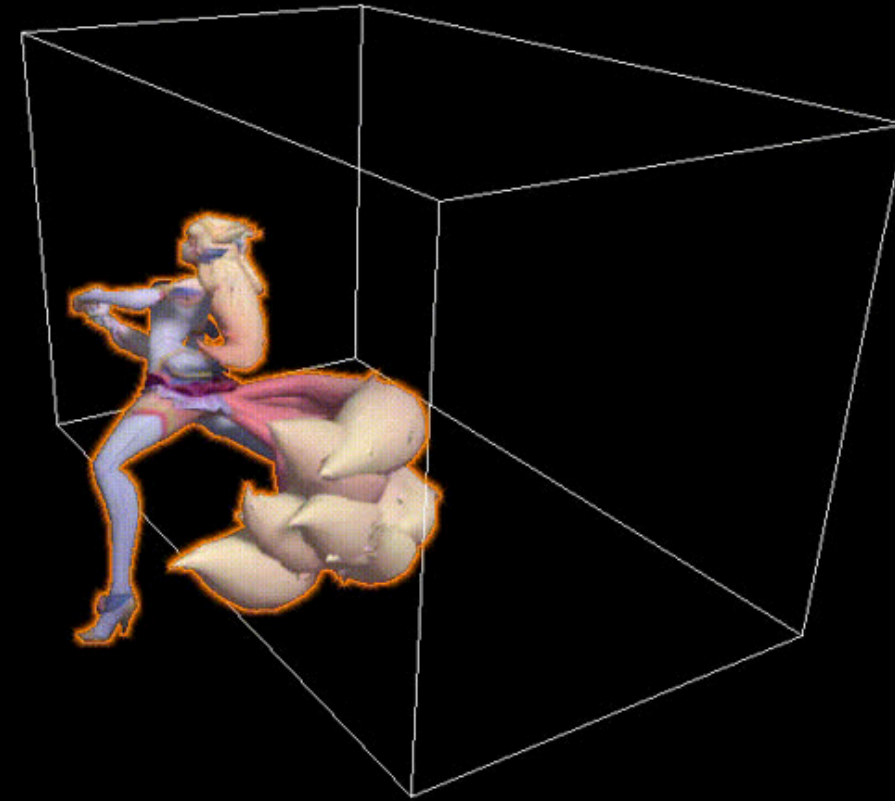
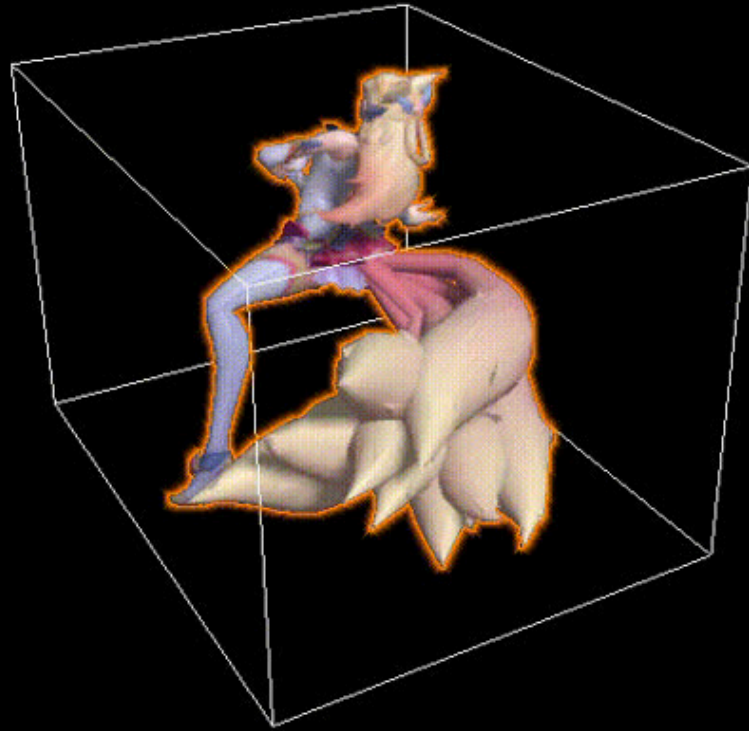
# CPU vs GPU Skinning

▶ 각 정점 변형을 **CPU**가 하는가, **GPU vertex shader**가 하는가

| 구분              | CPU Skinning                  | GPU Skinning                  |
|-----------------|-------------------------------|-------------------------------|
| 처리 방식           | <b>CPU</b> 가 변형된 정점 좌표를 직접 계산 | <b>GPU</b> 가 본 행렬과 가중치로 정점 계산 |
| 매 프레임<br>전송 데이터 | 변형된 정점 버퍼 갱신                  | 본 행렬 배열 전송                    |
| 장점              | 결과를 CPU에서 바로 활용 가능            | 대량 정점 처리에 유리                  |
| 단점              | 정점 버퍼 갱신 비용 큼                 | 변형 결과를 CPU에서 바로 쓰기 어려움        |

# CPU vs GPU Skinning

▶ CPU Skinning은 언제 사용할까?



결과가 GPU 안에서만 존재하므로  
Bounding Box가 갱신되지 않음

# CPU vs GPU Skinning

(Ahri mesh 30개의 recall animation sequence)

|                                            | CPU Skinning           | GPU Skinning         |
|--------------------------------------------|------------------------|----------------------|
| FPS                                        | 82.2 (12.16 ms)        | 338.4 (2.96 ms)      |
| CPU Anim/Pose                              | 1.790 / 0.196 ms       |                      |
| CPU Skin                                   | 5.176 ms               | 0.000 ms             |
| CPU VB Upload                              | 2.791 ms / 24547.85 KB | 0.000 ms / 0.00 KB   |
| GPU Bone CB Upload                         | 0.000 ms / 0.00 KB     | 0.049 ms / 960.47 KB |
| GPU Work<br>(vertex count * avg influence) | 0                      | 3799620              |

▶ 82.2fps → 338.4fps 약 4.1배 증가

# Reflection

▶ C++ 객체를 런타임에 읽고 조작할 수 있도록 하는 메타데이터 시스템



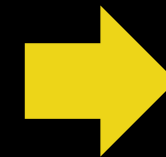
## C++ 소스코드

C++ 소스 코드에 UCLASS(),  
UPROPERTY() 등의 마커 작성



## Python 파서

메타데이터를 기록하고,  
레지스트리에 등록하는  
gen.cpp 코드 자동 작성



## 런타임 리플렉션

직렬화, 복사, 에디터에 노출할 때  
메타데이터 기반으로 찾아가서  
메모리를 읽고 씀



# Reflection

▶ 클래스에 프로퍼티의 설계를 등록하고, 레지스트리에 클래스의 설계를 등록한다

프로퍼티별  
메타데이터 생성

```
struct Z_Construct_UClass_USceneComponent {  
    static void RegisterRuntimeProperties(_UClass* Class) {  
        if (!Class) { return; }  
        Class->AddProperty(FProperty(FPropertyParams{  
            "RelativeLocation", "Location", nullptr, EPropertyType::Struct,  
            EPropertyFlags::Read | EPropertyFlags::Write | EPropertyFlags::Edit | EPropertyFlags::LuaReadWrite,  
            offsetof(USceneComponent, RelativeLocation), sizeof(((USceneComponent*)nullptr)->RelativeLocation),  
            0.0f, 0.0f, 0.1f, nullptr, nullptr, EObjectReferenceKind::None, nullptr, nullptr, nullptr, nullptr,  
            FVector::StaticStruct(), "FVector"  
        }));  
    }  
};
```

클래스에  
등록

# Reflection

▶ 클래스에 프로퍼티의 설계를 등록하고, 레지스트리에 클래스의 설계를 등록한다

```
UClass* USceneComponent::StaticClass()
{
    static UClass Class(
        "USceneComponent", UActorComponent::StaticClass(), sizeof(USceneComponent),
        CF_Component | CF_SpawnableComponent,
        []() -> UObject* { return CreateReflectedObject<USceneComponent>(); },
        "Scene Component", "Basic");

    static bool bRegistered = false;
    if (!bRegistered)
    {
        bRegistered = true;
        FReflectionRegistry::Get().RegisterUClass(&Class);
        Z_Construct_UClass_USceneComponent::RegisterRuntimeProperties(&Class);
    }
    return &Class;
}
```

클래스의  
메타데이터  
생성/반환

```
struct Z_AutoRegister_UClass_USceneComponent {
    Z_AutoRegister_UClass_USceneComponent() {
        USceneComponent::StaticClass();
    }
};
```

레지스트리에  
클래스 등록

# FProperty

▶ 개별 프로퍼티는 객체 주소 + 프로퍼티 Offset으로 메모리에 접근한다

```
struct FProperty
{
    const char* Name = "Hp";
    size_t Offset = 4;
    size_t Size = 4;
};
```

프로퍼티 메타데이터

객체 주소 + Offset



$0x1000 + 4 = 0x1004$

Player (0x1000)

|        |         |
|--------|---------|
| 0x1000 |         |
| 0x1004 | HP 실제 값 |
| 0x1008 |         |
| 0x100C |         |

런타임 메모리 블록



시연